Davante

5

ENTORNOS DE DESARROLLO

Instalación y explotación de entornos de desarrollo

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Instalación y configuración de entornos de desarrollo 4

/ 3. Eclipse, visual studio y netbeans 4

/ 4. Caso práctico 1: “IDE ligero” 5

/ 5. Actualizaciones y documentación 6

/ 6. Módulos adicionales y herramientas case 6

/ 7. Caso práctico 2: “Autogeneración de código” 7

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer el concepto de herramienta CASE.

Conocer las características para la elección de un IDE.

Conocer las principales características del IDE Eclipse y Visual Studio, y trabajar con el IDE NetBeans.

Conocer los requisitos de documentación.

Comprender la diferencia entre un IDE y un editor de texto

/ 1. Introducción y contextualización práctica

El desarrollo software es una tarea que cada vez posee mayor soporte a través de las herramientas que se van desarrollando continuamente. Es por ello por lo que existe un gran número de entornos de desarrollo en el mercado y debemos ser capaces de poder discernir cuál de ellos se adapta mejor a las necesidades de nuestros trabajos.

En este tema veremos las principales características de uno de los principales IDEs existentes como es Eclipse. También presentaremos las principales características de uno de los editores de texto más potentes en el mercado como es Visual Studio Code. Además, veremos las características que se esperan que tengamos en cuenta a la hora de elegir un IDE como, por ejemplo, su capacidad de ser ampliado a través de la utilización de plugins, sistema de actualización, etc.

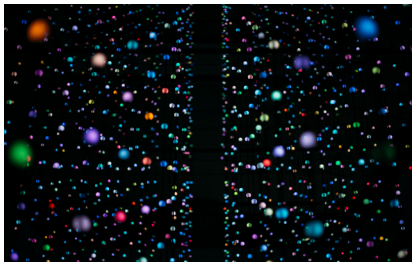


Fig. 1. La elección de un IDE y su configuración tienen múltiples posibilidades

Presentaremos el concepto de herramientas CASE y su utilización en el mundo del desarrollo software para apoyar las diferentes fases del ciclo de vida que tiene lugar cuando desarrollamos una aplicación.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.

Audio intro. ". Instalación de un IDE para el desarrollo web "

https://bit.ly/3kzaBAp

/ 2. Instalación y configuración de entornos de desarrollo

Para poder realizar cualquier desarrollo software es necesario disponer de un entorno de desarrollo que esté bien configurado e instalado en nuestro sistema.

Normalmente, la puesta en marcha de un entorno de desarrollo suele ser una tarea sencilla, aunque existen algunos entornos que requieren de una configuración especial. Además, muchas veces, la configuración del entorno de desarrollo va a depender del sistema operativo sobre el que estamos trabajando.

A la hora de elegir un entorno desarrollo para su instalación, tenemos que realizarnos algunas preguntas como, por ejemplo, cuál es su curva de aprendizaje, si es intuitivo, etc. En la actualidad, la mayor parte de los sistemas de desarrollo suelen tener características similares y su utilización suele ser semejante.

Para poder instalar un entorno de desarrollo, generalmente lo tenemos que descargar de la página de oficial del desarrollador. Una vez que se ha descargado el instalable podremos proceder a su instalación en nuestro sistema. Es importante que, como desarrolladores, nos sintamos cómodos a la hora de trabajar con un entorno de desarrollo. Para ello, es necesario realizar configuraciones iniciales que adapten el sistema a nuestros hábitos. Por ejemplo, suele ser recomendable establecer los códigos resumidos para determinadas acciones.

Más allá de una configuración básica, es recomendable realizar una personalización determinada, por ejemplo, ajustando el tamaño de la fuente del editor, colores de las palabras reservadas, etc.

Una vez que hemos realizado la configuración básica y personalización, sería recomendable instalar todos los módulos necesarios en nuestro nuevo IDE. Por ejemplo, nuestra empresa puede requerir la utilización de ciertos componentes y plugins que deben instalarse en el nuevo entorno.



Fig. 2. La instalación del IDE y sus componentes

Audio 1. "Desinstalación de plugins"

https://bit.ly/2XU6B3L

/ 3. Eclipse, visual studio y netbeans

Eclipse es un IDE gratuito y de código abierto que se usa para el desarrollo de aplicaciones desde hace mucho tiempo. Se trata de un espacio de trabajo fácilmente extensible gracias a la utilización de **plugins** que permiten personalizar el entorno. Eclipse está escrito principalmente en Java, siendo éste su principal lenguaje de desarrollo.

Sin embargo, Eclipse se puede utilizar para desarrollar en otros lenguajes a través de la instalación también de **plugins**. Por ejemplo, se permite el desarrollo en ADA, C/C++, **Clojure**, JavaScript, etc. Como se puede observar, Eclipse es capaz de soportar gran cantidad de lenguajes, convirtiéndose así en un IDE multilenguaje de propósito general.

La primera versión de Eclipse fue lanzada en el año 2004 y su última versión es de 2020. En todo este periodo Eclipse ha ido ampliando su funcionalidad, incluyendo diferentes opciones para mejorar su compatibilidad y estabilidad. Se puede utilizar este entorno para diseñar e implementar aplicaciones gráficas de escritorio utilizando plugins para AWT y Swing.

Uno de los grandes logros de Eclipse, es que se convirtió en el IDE oficial de Google para desarrollo de aplicaciones Android, aunque hoy en día, ya no lo es.

Visual Studio Code (VSC) es un editor de código fuente desarrollado por Microsoft para diferentes sistemas operativos como Windows, MacOs y Linux. Hay que destacar que, a diferencia de Eclipse, VSC no es un IDE para el desarrollo software, sino un editor de código fuente bastante avanzado.

La principal característica de este editor es que permite ampliar su funcionalidad significativamente a través de la instalación de diferentes **plugins**. Es un editor gratuito y de código abierto que incluye grandes características de personalización. Añadir nuevos **plugins** al editor es bastante sencillo, pues incorpora un gestor para ello.

Una de sus principales características es que es compatible con un gran número de lenguajes de programación que puede ser ampliado a través del sistema de personalización. Algunos de los lenguajes reconocidos son C/C++, C#, CSS, HTML, Java, Kotlin, PHP, etc.

Video 1. "Instalación de NetBeans "
https:/bit.ly/2MLXpdK

/ 4. Caso práctico 1: "IDE ligero"

Planteamiento. Nuestra empresa posee unos recursos limitados para la compra de nuevos equipos y por lo tanto se nos pide que utilicemos herramientas ligeras que puedan aumentar la productividad del desarrollador pero que no sobrecarguen la máquina con el fin de evitar la compra masiva para cada trabajador.

Nudo. ¿Cómo podrías afrontar esta situación?

Desenlace. Como hemos visto en el tema cada vez existe un mayor número de herramientas de desarrollo con el fin de aumentar la productividad de los programadores. Sin embargo, al tratarse de entornos integrados suelen realizar grandes tareas y por lo tanto suelen requerir grandes recursos hardware.

Es cierto que existen IDEs que son más ligeros unos que otros, pero en general suelen requerir bastante recursos. Por ejemplo, Eclipse podría ser un IDE ligero, pero a medida que ha ido evolucionando, cada vez requiere mayores recursos de la máquina.

En el caso del desarrollo móvil, existen IDEs como Android Studio que requiere gran cantidad de memoria RAM y una tarjeta gráfica potente que cubra todas las necesidades. En el caso del desarrollo para Apple es necesario disponer del IDE XCode que requiere también grandes recursos hardware.



Fig. 3. Tenemos que utilizar IDEs que se ajusten a los recursos de nuestras máquinas

En nuestra situación, la empresa nos pide que utilicemos herramientas ligeras que permiten aumentar la productividad. Una posibilidad sería utilizar editores de texto avanzados que permitan incluir nuevas funcionalidades según requieran los desarrolladores. Por ello, una posible solución sería utilizar Visual Studio Code. Aunque no sería la mejor solución para un programador, sería una solución que satisfaría las necesidades de la empresa hasta que pueda adquirir nuevo equipamiento en el que podamos instalar IDE acorde con nuestras necesidades.

/ 5. Actualizaciones y documentación

A. **Configuración de actualizaciones**

Una de las características principales del software que se desarrolla en la actualidad es que puede ser fácilmente actualizado. Bajo esta premisa, los IDEs y/o editores de códigos deberían ofrecer las mismas posibilidades a los programadores, pues se trata de herramientas creadas por desarrolladores para desarrolladores. Por ello, las principales herramientas de desarrollo que tenemos en la actualidad son fácilmente actualizable a través de su propia interfaz gráfica, tal y como hemos visto en el vídeo 1.

A la hora de elegir un IDE de desarrollo para nuestra empresa es importante tener en cuenta si ese entorno cuenta con un sistema de actualización y la frecuencia de éstas. Por ejemplo, si encontramos un IDE que es muy potente pero su frecuencia de actualización es muy baja, puede indicarnos que el equipo que está trabajando no es suficientemente grande y en poco tiempo dejará de ser efectivo.

B. **Acceso A Documentación**

El acceso a la documentación suele ser uno de los aspectos que obviamos cuando realizamos la elección de un IDE. Sin embargo, cuando empezamos a trabajar con él de manera periódica pueden existir tareas que no sepamos cómo realizar y por lo tanto debemos consultar la documentación. Si ésta no existe, tendremos un gran problema para poder encontrar las nuevas funcionalidades o aquellas que necesitemos utilizar.

Nótese que, un IDE es una herramienta bastante potente que ofrece un sinfín de posibilidades y que si no está correctamente documentada puede que deje de ser utilizada, ya que puede suponer una disminución de la productividad en los desarrolladores.

Además de estar documentado, un IDE debe proporcionar las herramientas necesarias para que el desarrollador pueda crear la documentación del propio software que está desarrollando. Al tratarse de un entorno integrado, debe incluir también esta fase del desarrollo. Existen lenguajes como Java que permiten introducir la documentación en el código fuente fácilmente a través (en el caso de Java) de JavaDoc.



Fig. 4. Un IDE debe ser fácilmente actualizable y estar preparado para documentar

/ 6. Módulos adicionales y herramientas case

A. **Instalación De Módulos Adicionales**

Como hemos visto anteriormente, uno de los requisitos más relevantes a la hora de elegir un nuevo IDE es que éste posea un sistema de extensión mediante plugins suficientemente potente. En la actualidad, la mayor parte de los IDEs ofrecen la posibilidad de extender la herramienta a través de pequeños módulos de personalización.

Cuando trabajamos con un IDE necesitamos que el flujo de trabajo sea suficientemente cómodo y se adapte a nuestras necesidades. Por esta razón, una característica relevante para un IDE es que sea capaz de desplegar un entorno de pruebas fácilmente. De esta forma, el programador no tiene que realizar tareas adicionales para probar la aplicación que está desarrollando.

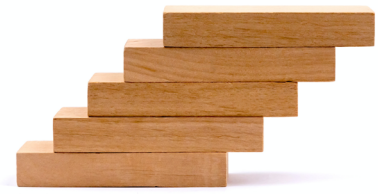


Fig. 5. Un IDE debe ofrecer diferentes módulos para funcionar

B. **Herramientas Case En El Desarrollo**

Las herramientas CASE (Computer Aided Software Engineering), son aplicaciones que dan soporte al desarrollador con el fin de aumentar su productividad en el desarrollo software. Estas herramientas pueden ayudar en todo el ciclo de vida del software en tareas relacionadas con el diseño, implementación automática del código, detección de errores, etc.

Como podemos deducir, las herramientas de este tipo suponen una automatización del desarrollo software que se puede traducir en una mejor calidad de éste. Al permitir la autogeneración de código, hay que destacar que el código autogenerado suele tener mayores dependencias del código escrito por los seres humanos. Además, este tipo de código, al ser genérico, suele tener menor calidad.

Aunque profundizaremos en ellas en el tema 7, de manera general podemos decir que las herramientas CASE se componen de los diferentes elementos: repositorio, sistema gestor de base de datos, metamodelo e interfaz de usuario.

Video 2. "Utilizando el IDE Netbeans"
https:/bit.ly/2zslu0L

/ 7. Caso práctico 2: "Autogeneración de código"

Planteamiento. Nuestro jefe contacta con uno de nuestros mayores clientes y tras conocer el concepto de herramientas CASE, le propone realizar un proyecto bastante grande en un tiempo muy corto. Cuando nos pide que revisemos el proyecto nos dice que como el código es autogenerado podemos tener el producto antes de tiempo.

Nudo. ¿Qué harías en esta situación? ¿Tiene sentido lo que nos propone?

Desenlace. Según hemos visto en el tema, las herramientas CASE ofrecen grandes posibilidades a los desarrolladores con el fin de facilitar fases como análisis, diseño, implementación, etc.

En nuestro caso nos están pidiendo que generemos código automático a través de estas herramientas, lo cual es posible, pues estas herramientas están diseñadas en parte para ello. Sin embargo, hay que tener en cuenta que el código que genera este tipo de herramienta es bastante limitado y suele requerir la intervención humana en muchas ocasiones.

Teniendo esto en mente, es fácilmente comprensible que tengamos que realizar ajustes en el código que, en ocasiones, tendremos que realizar desde cero.

Podemos decir que las herramientas CASE generan un código que no es de calidad y normalmente es poco eficiente. Plantearse realizar todo el desarrollo de una aplicación con este tipo de herramienta no es realista, pues en ocasiones va a requerir más tiempo que si lo desarrollasen seres humanos. Lo que sí tiene sentido es utilizar estas herramientas para tareas muy concretas como para la autogeneración de código en contextos limitados.



Fig. 6. Es necesario examinar con detalle el código autogenerado

/ 8. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema hemos presentado los principales aspectos que hay que tener en cuenta a la hora de **elegir un entorno de desarrollo integrado** para implementar nuestro componente software.

Como hemos podido ver, existen **diferentes aspectos que hay que considerar** como puede ser la generación de documentación, la existencia de la propia documentación de la herramienta y de un sistema de actualización, el grado de personalización de la herramienta, etc. Aunque a priori estos aspectos parezcan que no tienen importancia, a medida que cogemos experiencia y determinados vicios a la hora de desarrollar aplicaciones, se suelen echar en falta determinados aspectos cuando cambiamos de un entorno a otro.

Nótese que, como desarrolladores es muy habitual que cambiemos de un entorno de desarrollo a otro en cuestión de horas o días, dependiendo del número de proyectos que tengamos en nuestro equipo o empresa.

Resolución Del Caso Práctico De La Unidad

A la hora de instalar un entorno de desarrollo no podemos dar una única respuesta correcta. Actualmente, en el desarrollo web existe un amplio número de tecnologías y un amplio número de entornos que poseen unas características diferentes y que se adaptan mejor o peor a los requisitos del usuario.

El único requisito que nos pide nuestro vecino es que sea una aplicación web que pueda incorporar compras. Esta funcionalidad se puede realizar prácticamente con cualquier lenguaje y entorno, no requiriendo ningún lenguaje con una capacidad especial. Quizás, lo más sencillo sería utilizar un entorno de desarrollo como Visual Studio Code y añadir los plugin necesarios para el desarrollo web.

Uno de los lenguajes más utilizados para el desarrollo de páginas web en el lado del servidor hoy en día es PHP. Dado que los servidores de PHP son más económicos de mantener, podría ser una opción, pues además es un lenguaje muy fácil de aprender para los desarrolladores.



Fig. 7. PHP y Visual Studio son apropiados para el desarrollo web

/ 9. Bibliografía

Boudreau, T. (2003). NetBeans: the definitive guide. Beijing Cambridge Mass: O'Reilly.

Burnette, E. (2005). Eclipse IDE: pocket guide. Sebastopol, CA: O'Reilly.

Hagos, T. (2019). Android Studio IDE Quick Reference: a Pocket Guide to Android Studio Development. Berkeley, CA: Apress Imprint Apress.